المدرسة العليا للتكنولوجيا
Ecole Supérieure de Technologie
Sidi Bennour - سيدي بنور

Rapport du projet :

Reconnaissance de postures de Yoga (HOG + KNN)

Encadré par :
Pr. SMMAA

Réalisé par :
MEFTAH Nouhaila
CHENNOUFI Imane
DRHOURHI Omaima

Année universitaire : 2025/2026

Table des matières

TABLE DES FIGURES : 6

CHAPITRE I: INTRODUCTION 7

1. Contexte et motivation : 7

2. Problématique : 7

3. Objectifs du projet : 8

4. Portée du projet : 8

5. Solution propose : 9

CHAPITRE II: LES ALGORITHMES 11

1. L'ALGORITHME KNN : 11

1.1 Définition et rôle du KNN : 11

1.1.1 Définition : 11

1.1.2 Son rôle : 11

1.2 Implémentation du KNN dans les Scripts : 11

1.2.1 Fichier src/pipeline.py (Le "Moteur" du projet) : 11

1.2.2 Fichier src/main.py (Le Script du Terminal) : 12

1.3 Chronologie du KNN (via le Terminal) : 13

2. L'ALGORITHME SVM : 14

2.1 Définition et rôle du SVM : 14

2.1.1 Définition : 14

2.1.2 Son rôle dans notre projet : 14

2.2 Implémentation du SVM dans les scripts : 15

2.2.1 Création du modèle SVM : 15

2.2.2 Utilisation dans le script principal 16

2.3 Chronologie du fonctionnement du SVM : 16

2.3.1 Chargement des données : 16

2.3.2 Extraction des caractéristiques 17

2.3.3 Division du dataset : 17

2.3.4 Entraînement du modèle : 17

2.3.5 Prédiction : 17

2.3.6 Évaluation du modèle : 18
2.4 Résultat obtenu avec SVM : 18
3. L'algorithme HOG : 18
3.1 Définition et Rôle : 18
3.2.1 Définition : 18
3.2.2 Rôle HOG : 19
3.2 Implémentation du HOG dans les scripts : 19
3.2.1 Création du HOG : 20
3.2.2 Extraction des caractéristiques HOG 20
3.2.3 Utilisation dans le script principal : 21
3.3 Chronologie du fonctionnement du HOG : 21
3.3.1 Chargement des images : 21
3.3.2 Prétraitement des images : 22
3.3.3 Extraction des caractéristiques HOG : 22
3.3.4 Création du dataset : 23
CHAPITRE III: L'ARCHITECTURE DU CODE : 24
1. Introduction 24
2. Structure générale du projet 24
3. Dossier data : 25
3.1 Raw : 25
4. Dossier src : 25
4.1 main.py: 26
4.2 pipeline.py : 26
4.3 utils.py : 26
4.3 remove_background.py : 26
4.4 web_app.py : 27
4.5 Dossier results : 27
4.5 Fichiers de configuration 27
CHAPITRE IV: RESULTAT ET CONCLUSION : 29
1. Résultats du Terminal (en exécutant src/main.py) : 29

2. Page Web (en exécutant streamlit run src/web_app.py) : 30
3. Conclusion : 32

TABLE DES FIGURES :

Figure 1: Code d'importation de la classe KNN 12
Figure 2: Code de la création du modèle KNN 12
Figure 3: Création du modèle SVM 15
Figure 4: Entraînement du modèle : 17
Figure 5: Prédiction : 17
Figure 6: Création du HOG : 20
Figure 7: Chargement des images : 22
Figure 8: Prétraitement des images : 22
Figure 9: Extraction des caractéristiques HOG : 23
Figure 10: Création du dataset : 23
Figure 11: Résultat du terminal 29
Figure 12: Résultat de l'entrainement sur la page web 31
Figure 13: Les diagrammes 31
Figure 14: Prédiction des poses 32

CHAPITRE I: INTRODUCTION

1. Contexte et motivation :

Avec le développement des technologies numériques et de l'intelligence artificielle, plusieurs domaines de la vie quotidienne ont bénéficié d'applications intelligentes. Parmi ces domaines, le domaine du fitness et du bien-être, notamment la pratique du yoga, a connu une grande popularité ces dernières années.

Le yoga est une activité physique et mentale qui aide à améliorer la flexibilité, la concentration et la santé générale. Cependant, l'exécution correcte des poses de yoga est essentielle afin d'éviter les blessures et d'obtenir les bénéfices attendus. Dans ce contexte, l'utilisation de techniques d'intelligence artificielle (IA) et de machine learning (ML) permet de développer des systèmes capables d'analyser automatiquement les mouvements du corps et de reconnaître différentes poses.

Ainsi, la reconnaissance automatique des poses de yoga représente un défi intéressant dans le domaine de la vision par ordinateur, car elle nécessite l'analyse des formes et des orientations du corps humain dans les images.

2. Problématique :

Lors de la pratique du yoga, il peut être difficile pour une personne de savoir si une pose est correctement exécutée, surtout en l'absence d'un coach ou d'un instructeur. Les erreurs dans l'exécution peuvent réduire l'efficacité de l'exercice ou provoquer des blessures.

De plus, les poses de yoga peuvent varier d'une personne à une autre en raison des différences de morphologie, d'angle de prise de vue ou de fond d'image. Cela rend la reconnaissance automatique des poses plus complexe.

Ainsi, la problématique principale de ce projet est de déterminer comment concevoir un système capable de reconnaître automatiquement différentes poses de yoga à partir d'images, tout en assurant une bonne précision malgré la variabilité des images.

3. Objectifs du projet :

L'objectif principal de ce projet est de développer un système de classification automatique des poses de yoga à partir d'images.

Plus précisément, ce projet vise à :

- Extraire les caractéristiques des images à l'aide du descripteur HOG (Histogram of Oriented Gradients).
- Utiliser des algorithmes de machine learning pour classifier les poses de yoga.
- Comparer les performances de deux algorithmes de classification : SVM (Support Vector Machine) et KNN (K-Nearest Neighbors).
- Étudier l'impact du fond de l'image sur la précision de la classification.

4. Portée du projet :

Dans le cadre de ce projet, l'étude se concentre sur la reconnaissance automatique d'un ensemble limité de poses de yoga. Plus précisément, le système développé est conçu pour identifier cinq poses de yoga principales, qui sont parmi les plus connues et les plus pratiquées dans les exercices de yoga. Ces poses sont les suivantes :

- Warrior : la pose Warrior est une posture de yoga où une jambe est placée en avant fléchie et l'autre en arrière tendue, avec les bras levés ou étendus. Elle permet de renforcer les jambes et d'améliorer l'équilibre et la stabilité du corps.
- Downward Dog (Downdog) **:** la pose Downward Dog est une posture où le corps forme un triangle inversé, avec les mains et les pieds au sol et les hanches levées. Elle permet d'étirer les muscles du dos, des bras et des jambes.

- Goddess : la pose Goddess est une posture où les jambes sont écartées et légèrement fléchies, avec les bras levés et les coudes pliés. Elle aide à renforcer les jambes et à améliorer la stabilité.
- Plank : la pose Plank est une posture de gainage où le corps reste droit et aligné, soutenu par les bras et les pieds. Elle renforce les muscles du tronc.
- Tree : la pose Tree est une posture d'équilibre où une jambe reste au sol tandis que l'autre est placée contre la jambe opposée, avec les bras levés. Elle améliore l'équilibre et la concentration.

Les images utilisées proviennent d'un dataset disponible sur Kaggle, contenant des images avec fond et sans fond. Les caractéristiques des images sont extraites à l'aide du descripteur HOG, qui permet de représenter les orientations des gradients dans l'image.

5. Solution propose :

Pour répondre à la problématique posée, nous proposons de concevoir un système de reconnaissance automatique des poses de yoga basé sur des techniques de vision par ordinateur et de machine learning. L'objectif est d'analyser des images contenant différentes poses de yoga et de déterminer automatiquement à quelle classe appartient chaque pose.

Le système proposé repose sur un pipeline de traitement d'images et de classification, composé de plusieurs étapes successives permettant de transformer les images brutes en informations exploitables par les algorithmes d'apprentissage automatique.

La première étape consiste en un prétraitement des images. Cette étape vise à préparer les images avant l'analyse, notamment en les redimensionnant, en les convertissant en niveaux de gris et en normalisant certaines caractéristiques afin de garantir une meilleure qualité des données utilisées pour l'apprentissage.

Ensuite, les caractéristiques importantes des images sont extraites à l'aide du descripteur HOG (Histogram of Oriented Gradients). Cette méthode permet de capturer la structure et l'orientation des contours présents dans l'image. Elle est particulièrement adaptée à la détection de formes et de postures humaines, car elle met en évidence les gradients et les orientations des différentes parties du corps.

Une fois les caractéristiques extraites, elles sont utilisées comme entrée pour des algorithmes de classification supervisée. Dans ce projet, deux algorithmes sont utilisés et comparés :

- SVM (Support Vector Machine) : connu pour sa capacité à trouver une frontière optimale entre différentes classes de données.
- KNN (K-Nearest Neighbors) : un algorithme simple basé sur la proximité entre les données.

Ces deux modèles sont entraînés sur les données extraites afin de prédire la pose de yoga correspondant à chaque image.

Après l'entraînement des modèles, une phase d'évaluation est réalisée afin d'analyser les performances des algorithmes. Cette analyse permet de comparer la précision des modèles, d'identifier leurs points forts et leurs limites, et d'étudier l'influence du fond de l'image sur la qualité de la classification.

Enfin, pour faciliter l'interprétation et la visualisation des résultats, un dashboard web interactif est développé. Ce tableau de bord permet d'afficher les performances des modèles, de comparer les résultats obtenus avec différentes configurations et de présenter les résultats de manière claire et intuitive.

CHAPITRE II: LES ALGORITHMES

1. L'ALGORITHME KNN :

1.1 Définition et rôle du KNN :

1.1.1 Définition :

L'algorithme KNN (K-Nearest Neighbors ou K-Plus Proches Voisins) est un algorithme d'apprentissage supervisé. Pour faire une prédiction, il regarde les "K" points de données les plus similaires (ses "plus proches voisins") dans le jeu de données d'entraînement. Le nouveau point de données est alors assigné à la classe la plus fréquente parmi ces K voisins. La "proximité" est généralement calculée à l'aide d'une mesure de distance (par exemple, la distance euclidienne).

1.1.2 Son rôle :

Dans notre projet, le KNN est utilisé comme un classifieur pour identifier les postures de yoga à partir des images. Il sert d'alternative au classifieur SVM que vous utilisiez déjà. Le projet est maintenant capable d'entraîner et de comparer les deux modèles pour déterminer lequel est le plus performant pour cette tâche.

1.2 Implémentation du KNN dans les Scripts :

L'algorithme a été implémenté principalement dans le fichier src/pipeline.py et est utilisé par src/main.py pour l'entraînement via le terminal.

1.2.1 Fichier src/pipeline.py (Le "Moteur" du projet) :

C'est dans ce fichier que le modèle KNN est créé et entraîné.

- Importation de la classe KNN : Tout d'abord, la classe nécessaire est importée depuis la bibliothèque scikit-learn.

```python
from sklearn.neighbors import KNeighborsClassifier
```

Figure 1: Code d'importation de la classe KNN

- Création du modèle KNN : La fonction _build_model a été crée pour pouvoir construire soit un modèle SVM, soit un modèle KNN. Voici la partie qui nous intéresse :

```python
def _build_model(
    classifier_name: str,
    random_state: int = 42,
    knn_neighbors: int = 5,
) -> Any:
    normalized = normalize_classifier_name(classifier_name)

    if normalized == "svm":
        return make_pipeline(
            StandardScaler(),
            SVC(kernel="linear", C=1.0, probability=True, random_state=random_state),
        )

    # Création du pipeline pour le KNN
    return make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=knn_neighbors, weights="distance"),
    )
```

Figure 2: Code de la création du modèle KNN

  - make_pipeline(...) : Crée une séquence d'opérations. D'abord, StandardScaler() normalise les données, puis KNeighborsClassifier(...) est appliqué.
  - n_neighbors=knn_neighbors : C'est le "K" de KNN. Il est défini à 5 par défaut. Le modèle regardera donc les 5 voisins les plus proches pour faire sa prédiction.
  - weights="distance" : Au lieu de donner un vote égal à chaque voisin, ce paramètre donne plus d'importance aux voisins les plus proches. Un voisin très proche aura plus d'influence sur le vote qu'un voisin plus éloigné.

1.2.2 Fichier src/main.py (Le Script du Terminal) :

Ce script utilise le moteur pipeline.py pour lancer les entraînements et sauvegarder les résultats. Il ne contient pas la logique du KNN lui-même, mais il lui dit quand et comment s'exécuter.

1.3 Chronologie du KNN (via le Terminal) :

Voici ce qui se passe étape par étape lorsque vous lancez la commande python src/main.py.

- Lancement : Vous exécutez le script. La fonction main() démarre et appelle la fonction run_background_comparison du fichier pipeline.py. Elle lui demande de lancer les expériences pour les classifieurs par défaut, qui sont ["svm", "knn"].
- Préparation des données :
  - Pour une expérience donnée (par exemple, avec les images sans fond), les images sont chargées.
  - Les caractéristiques HOG (Histogram of Oriented Gradients) sont extraites de chaque image. Cela transforme chaque image en un vecteur de nombres que le modèle peut comprendre.
  - L'ensemble de données est divisé en un jeu d'entraînement (pour apprendre) et un jeu de test (pour évaluer).
- Entraînement du modèle KNN :
  - La fonction train_experiment est appelée avec le paramètre classifier_name="knn".
  - Un nouveau modèle KNN est créé via la fonction _build_model.
  - La fonction model.fit(x_train, y_train) est exécutée. À ce moment, le modèle KNN "apprend" en mémorisant simplement toutes les caractéristiques HOG (x_train) et les étiquettes de postures correspondantes (y_train).
- Prédiction et Évaluation :
  - La fonction model.predict(x_test) est appelée.
  - Pour chaque image du jeu de test, le modèle KNN recherche les 5 images les plus similaires qu'il a vues pendant l'entraînement (ses "plus proches voisins").
  - Il prédit la posture en se basant sur un vote pondéré par la distance de ces 5 voisins.
  - Les prédictions sont comparées aux vraies étiquettes pour calculer l'accuracy (précision) et générer une matrice de confusion.
- Sauvegarde des Résultats :
  - Le script main.py récupère les résultats de l'expérience KNN (précision, matrice de confusion).
  - Il utilise ces données pour générer des graphiques avec matplotlib :

- Une image de la matrice de confusion pour le modèle KNN est sauvegardée dans results/plots/.
- Un graphique à barres comparant la précision du KNN à celle du SVM est également sauvegardé.

- Un tableau récapitulatif des résultats est affiché dans le terminal

2. L'ALGORITHME SVM :

2.1 Définition et rôle du SVM :

2.1.1 Définition :

L'algorithme SVM (Support Vector Machine) est un algorithme d'apprentissage supervisé utilisé pour les problèmes de classification et de régression.

Le principe du SVM est de trouver un hyperplan optimal qui permet de séparer les données appartenant à différentes classes avec la plus grande marge possible.

Les points de données les plus proches de cet hyperplan sont appelés vecteurs de support (Support Vectors).
Ces vecteurs sont les éléments les plus importants car ils déterminent la position de la frontière de décision.

Dans les problèmes de classification, le SVM cherche donc à maximiser la distance entre les classes, ce qui permet d'obtenir un modèle robuste et précis.

2.1.2 Son rôle dans notre projet :

Dans notre projet de reconnaissance des postures de yoga, l'algorithme SVM est utilisé comme classifieur principal.

Après l'extraction des caractéristiques des images à l'aide du descripteur HOG (Histogram of Oriented Gradients), ces caractéristiques sont utilisées comme entrée du modèle SVM.

Le rôle du SVM est donc de :

- analyser les vecteurs de caractéristiques HOG
- apprendre les différences entre les différentes postures
- prédire la posture correcte pour une nouvelle image

Le modèle SVM est également comparé avec l'algorithme KNN afin d'évaluer lequel offre les meilleures performances dans notre système.

2.2 Implémentation du SVM dans les scripts :

L'algorithme SVM est implémenté principalement dans le fichier :

src/pipeline.py

Ce fichier contient le code responsable de la création, de l'entraînement et de l'évaluation du modèle.

2.2.1 Création du modèle SVM :

Le modèle SVM est créé en utilisant la bibliothèque Scikit-learn**.**

Dans notre projet, nous utilisons un SVM linéaire avec normalisation des données.

Exemple de code :

```
make_pipeline(
            StandardScaler(),
            SVC(kernel="linear", C=1.0, probability=True, random_state=random_state),
```

Figure 3: Création du modèle SVM

Dans cette configuration :

- StandardScaler() **:** permet de normaliser les données pour améliorer les performances du modèle.
- SVC() : est la classe qui implémente l'algorithme Support Vector Machine.
- kernel="linear" : signifie que le modèle utilise un noyau linéaire pour séparer les données.
- C=1.0 **:** est un paramètre de régularisation qui contrôle le compromis entre précision et généralisation.

2.2.2 Utilisation dans le script principal

Le script principal :src/main.py, utilise le fichier pipeline.py pour lancer l'entraînement du modèle SVM.

Ce script permet de :

- charger les images
- extraire les caractéristiques HOG
- entraîner le modèle SVM
- tester la performance du modèle

2.3 Chronologie du fonctionnement du SVM :

Lorsque l'utilisateur exécute la commande suivante dans le terminal : python src/main.py

plusieurs étapes sont exécutées automatiquement.

2.3.1 Chargement des données :

Les images sont chargées depuis les dossiers :

data/raw
data/raw_sans_fond

Chaque dossier contient les différentes postures de yoga.

2.3.2 Extraction des caractéristiques

Chaque image est transformée en vecteur de caractéristiques grâce à l'algorithme HOG.

Le descripteur HOG analyse les gradients de l'image afin de capturer la forme globale du corps humain.

2.3.3 Division du dataset :

Les données sont divisées en deux ensembles :

- 80 % pour l'entraînement
- 20 % pour le test

Cette séparation permet d'évaluer la performance du modèle sur des données non vues.

2.3.4 Entraînement du modèle :

Le modèle SVM est entraîné avec la commande :

```
model.fit(x_train, y_train)
```

Figure 4: Entraînement du modèle :

Pendant cette étape, le SVM apprend à distinguer les différentes postures en construisant une frontière de décision optimale.

2.3.5 Prédiction :

Après l'entraînement, le modèle peut prédire la posture d'une nouvelle image :

```
model.predict(x_test)
```

Figure 5: Prédiction :

Pour chaque image test, le modèle détermine la classe la plus probable.

2.3.6 Évaluation du modèle :

Les prédictions sont comparées aux vraies classes afin de calculer plusieurs métriques :

- Accuracy
- Precision
- Recall
- F1-score

Une matrice de confusion est également générée afin d'analyser les erreurs de classification.

2.4 Résultat obtenu avec SVM :

Dans notre projet, le modèle SVM a obtenu les résultats suivants :

- Avec fond : 76.50 %
- Sans fond : 87.38 %

Ces résultats montrent que la suppression de l'arrière-plan améliore significativement la performance du modèle.

3. L'algorithme HOG :

3.1 Définition et Rôle :

3.2.1 Définition :

La méthode Histogram of Oriented Gradients (HOG) est une technique d'extraction de caractéristiques largement utilisée dans le domaine de la vision par ordinateur.

Elle permet de décrire la structure et la forme des objets présents dans une image en analysant les variations d'intensité des pixels, appelées gradients.

Le principe de HOG consiste à calculer les gradients de l'image, puis à regrouper leurs orientations dans des histogrammes. Ces histogrammes représentent les directions dominantes des contours et des formes dans différentes régions de l'image.

Cette méthode est particulièrement efficace pour détecter les formes du corps humain, car elle met en évidence les contours et les silhouettes, qui sont essentiels pour distinguer différentes postures.

3.2.2 Rôle HOG :

Dans notre projet de reconnaissance des postures de yoga, le descripteur HOG est utilisé pour extraire les caractéristiques importantes des images.

Son rôle est de transformer chaque image en vecteur de caractéristiques qui décrit la forme du corps humain.

Ces vecteurs sont ensuite utilisés comme entrée pour les algorithmes de classification, notamment :

- SVM (Support Vector Machine)
- KNN (K-Nearest Neighbors)

Ainsi, le HOG permet de :

- détecter les contours et la silhouette du corps
- représenter les images sous forme de données numériques exploitables
- améliorer la performance des modèles de classification

3.2 Implémentation du HOG dans les scripts :

Dans notre projet, l'implémentation du HOG est réalisée dans les scripts Python responsables du traitement des images.

Le code permettant d'extraire les caractéristiques HOG se trouve principalement dans :

src/pipeline.py

Ce fichier contient les fonctions utilisées pour :

- charger les images
- préparer les données
- extraire les caractéristiques HOG

3.2.1 Création du HOG :

Le modèle SVM est créé en utilisant la bibliothèque Scikit-learn. Dans notre projet, nous utilisons un SVM linéaire avec normalisation des données pour améliorer la performance sur les caractéristiques HOG.

Code d'implémentation du SVM dans pepline.py

```
def _build_model(
    classifier_name: str,
    random_state: int = 42,
    knn_neighbors: int = 5,
) -> Any:
    normalized = normalize_classifier_name(classifier_name)

    if normalized == "svm":
        return make_pipeline(
            StandardScaler(),
            SVC(kernel="linear", C=1.0, probability=True, random_state=random_state),
        )
```

Figure 6: Création du HOG :

3.2.2 Extraction des caractéristiques HOG

Pour chaque image, l'algorithme HOG calcule les gradients et génère un vecteur de caractéristiques.

Dans notre projet, nous utilisons la bibliothèque scikit-image pour appliquer le descripteur HOG.

Exemple de paramètres utilisés :

- Orientations = 9
  nombre de directions de gradients analysées

- pixels_per_cell = (8,8)
  taille des cellules utilisées pour calculer les histogrammes
- cells_per_block = (2,2)
  nombre de cellules utilisées pour la normalisation

Ces paramètres permettent de capturer efficacement les formes et les contours du corps humain dans les images.

3.2.3 Utilisation dans le script principal :

Le script principal du projet est :

src/main.py

Ce script permet de lancer l'ensemble du pipeline de traitement.

Il exécute les étapes suivantes :

- chargement des images
- extraction des caractéristiques HOG
- entraînement des modèles de classification
- évaluation des performances

3.3 Chronologie du fonctionnement du HOG :

Lorsque l'utilisateur exécute la commande suivante dans le terminal :

python src/main.py

plusieurs étapes sont exécutées automatiquement.

3.3.1 Chargement des images :

Les images sont chargées depuis les dossiers :

- data/raw
- data/raw_sans_fond

```python
def _load_grayscale_images(
    folder_path: Path,
    max_images: int | None = None,
    random_state: int = 42,
) -> list[np.ndarray]:
    """
    Charge toutes les images depuis un dossier (data/raw ou data/raw_sans_fond).
    Filtre les fichiers par extensions valides et trie par nom.
    """
    if not folder_path.exists():
        return []
```

Figure 7: Chargement des images :

Chaque dossier contient différentes postures de yoga.

3.3.2 Prétraitement des images :

Les images sont converties en niveau de gris puis redimensionnées afin d'obtenir un format uniforme.

```python
def _load_grayscale_images(
    folder_path: Path,
    max_images: int | None = None,
    random_state: int = 42,
) -> list[np.ndarray]:
    # ... (code de chargement ci-dessus) ...

    images: list[np.ndarray] = []
    for file_path in files:
        # PRÉTRAITEMENT : Conversion en niveaux de gris
        image = cv2.imread(str(file_path), cv2.IMREAD_GRAYSCALE)
        if image is None:
            continue
        # PRÉTRAITEMENT : Redimensionnement à 128x64 (format uniforme pour HOG)
        image = cv2.resize(image, (128, 64))
        images.append(image)

    return images
```

Figure 8: Prétraitement des images :

3.3.3 Extraction des caractéristiques HOG :

Chaque image est transformée en vecteur de caractéristiques grâce au descripteur HOG.

Cette étape permet de capturer la forme globale du corps humain dans chaque posture.

```python
def extract_hog_features(images):
    """Calcule les descripteurs HOG pour une liste d'images"""
    features_list = []
    hog_images_list = []  # Pour la visualisation

    for img in images:
        # EXTRACTION HOG : Calcul des caractéristiques
        features, hog_image = hog(img, orientations=9, pixels_per_cell=(8, 8),
                                  cells_per_block=(2, 2), visualize=True, feature_vector=True)
        features_list.append(features)
        hog_images_list.append(hog_image)

    return np.array(features_list), hog_images_list
```

Figure 9: Extraction des caractéristiques HOG :

3.3.4 Création du dataset :

Les vecteurs HOG sont regroupés pour former le dataset utilisé pour l'apprentissage automatique.

```python
def _build_dataset(
    dataset_path: Path,
    poses: list[str],
    max_images_per_class: int | None = None,
    random_state: int = 42,
) -> tuple[np.ndarray, np.ndarray, list[str], dict[str, int]]:
    """
    Construit le dataset en regroupant les vecteurs HOG de toutes les poses.
    """
    x_parts: list[np.ndarray] = []
    y_labels: list[str] = []
    valid_poses: list[str] = []
    class_counts: dict[str, int] = {}
```

Figure 10: Création du dataset :

Ces données sont ensuite utilisées par les modèles de classification comme SVM et KNN.

CHAPITRE III: L'ARCHITECTURE DU CODE :

1. Introduction

L'architecture du code représente l'organisation des différents fichiers et modules qui composent l'application. Une bonne architecture permet de faciliter la maintenance du projet, d'améliorer la lisibilité du code et de séparer les différentes fonctionnalités du système.

Dans notre projet de reconnaissance des postures de yoga, le code est organisé en plusieurs dossiers et fichiers, chacun ayant un rôle spécifique. Cette structure permet de séparer les données, le code source, les résultats et les livrables du projet.

2. Structure générale du projet

La structure globale du projet est organisée comme suit :

```
PROJET_YOGA_AI
│
├── data
│   ├── raw
│   └── raw_sans_fond
│
├── results
│   ├── models
│   └── plots
│
├── src
│   ├── main.py
│   ├── pipeline.py
│   ├── remove_background.py
│   ├── utils.py
│   └── web_app.py
│
├── requirements.txt
├── readme.md
└── .gitignore
```

Cette organisation permet de séparer clairement les différentes parties du projet.

3. Dossier data :

Le dossier data contient l'ensemble du dataset utilisé pour entraîner et tester les modèles de classification.

Il est composé de deux sous-dossiers :

3.1 Raw :

Ce dossier contient les images originales des différentes postures de yoga avec leur arrière-plan.

3.2 raw_sans_fond :

Ce dossier contient les images après suppression de l'arrière-plan. La suppression du fond permet d'améliorer l'extraction des caractéristiques et d'évaluer l'impact de l'arrière-plan sur les performances du modèle.

Les différentes classes utilisées dans le dataset sont :

- warrior
- downdog
- goddess
- plank
- tree

4. Dossier src :

Le dossier src contient le code source principal de l'application. Il regroupe les scripts responsables du traitement des images, de l'entraînement des modèles et de l'interface utilisateur.

4.1 main.py:

Ce fichier constitue le point d'entrée principal du programme. Il permet de lancer les expériences, entraîner les modèles et afficher les résultats obtenus.

Le script exécute les différentes étapes du projet :

- chargement des données
- extraction des caractéristiques
- entraînement des modèles
- évaluation des performances

4.2 pipeline.py :

Ce fichier représente le cœur du système. Il contient les fonctions permettant de :

- charger les images
- extraire les caractéristiques HOG
- entraîner les modèles SVM et KNN
- générer les matrices de confusion
- calculer les métriques de performance

Il permet également de comparer les performances des différents modèles.

4.3 utils.py :

Ce fichier contient plusieurs fonctions utilitaires utilisées dans le projet, notamment celles liées au traitement des images et à l'extraction des caractéristiques HOG. Ces fonctions permettent de simplifier et d'organiser le code.

4.3 remove_background.py :

Ce script permet de supprimer l'arrière-plan des images du dataset afin de générer un second jeu de données sans fond.

Cette étape permet d'améliorer la qualité des caractéristiques extraites et d'analyser l'impact de l'arrière-plan sur la précision du modèle.

4.4 web_app.py :

Ce fichier implémente une interface web interactive utilisant la bibliothèque Streamlit.

Cette interface permet :

- d'entraîner les modèles
- de visualiser les résultats
- de comparer les performances
- de tester la prédiction sur une nouvelle image

4.5 Dossier results :

Le dossier results contient les résultats générés par les expériences réalisées.

Il est composé de deux sous-dossiers :

- Models :

Ce dossier contient les modèles entraînés qui peuvent être réutilisés pour effectuer des prédictions.

- Plots :

Ce dossier contient les graphiques générés par l'application, notamment :

  - les matrices de confusion
  - les graphiques de comparaison des performances
  - les visualisations des résultats

4.5 Fichiers de configuration

- requirements.txt :

Ce fichier contient la liste des bibliothèques Python nécessaires pour exécuter le projet, comme

- scikit-learn
- opencv
- numpy
- matplotlib
- streamlit

➢ readme.md :

Ce fichier fournit une description générale du projet ainsi que les instructions nécessaires pour installer et exécuter l'application.

CHAPITRE IV: RESULTAT ET CONCLUSION :

1. Résultats du Terminal (en exécutant src/main.py) :

Lorsque vous exécutez le script principal via le terminal, celui-ci se concentre sur l'entraînement des modèles et la présentation d'un résumé concis des performances. Voici ce que vous verrez :

- Lancement de l'Analyse : Le script affichera un titre "YOGA AI - Comparaison HOG avec SVM et KNN".
- Tableau de Résultats : Un tableau récapitulatif sera imprimé, comparant les performances des modèles. Il contiendra des colonnes telles que :
  - Model : Le nom de la configuration (par exemple, "Avec Fond (Original) - SVM")
  - Experience : Indique si le modèle a été entraîné sur des images avec ou sans fond.
  - Classifier : Le type de modèle utilisé (SVM ou KNN).
  - Accuracy (%) : Le pourcentage de prédictions correctes sur l'ensemble de test.
- Gain de Performance : Le terminal affichera le gain de précision (en pourcentage) obtenu en supprimant l'arrière-plan des images pour chaque classifieur (SVM et KNN). Cela met en évidence l'impact du prétraitement.
- Sauvegarde des Graphiques : Des messages de confirmation vous informeront que plusieur s graphiques ont été sauvegardés dans le dossier results/plots/, incluant les matrices de confusion pour chaque modèle et des graphiques comparatifs.

En résumé, le terminal offre une vue rapide et quantitative de l'efficacité des modèles et de l'importance de la suppression de l'arrière-plan.

```
PS C:\Users\Nouhaila\Projet_Yoga_AI> python src/main.py
>>
YOGA AI - Comparaison HOG avec SVM et KNN
Corrupt JPEG data: premature end of data segment
Premature end of JPEG file
Corrupt JPEG data: premature end of data segment
Premature end of JPEG file

Resultats:
                    Model           Experience Classifier  Accuracy (%)  Train Samples  Test Samples
Avec Fond (Original) - SVM Avec Fond (Original)        SVM         74.65            864           217
Avec Fond (Original) - KNN Avec Fond (Original)        KNN         64.06            864           217
 Sans Fond (Traitee) - SVM  Sans Fond (Traitee)        SVM         81.78            853           214
 Sans Fond (Traitee) - KNN  Sans Fond (Traitee)        KNN         78.97            853           214

Gain sans fond par classifieur:
- SVM: +7.12%
- KNN: +14.92%
Matrice sauvegardee: C:\Users\Nouhaila\Projet_Yoga_AI\results\plots\confusion_avec_fond_original___svm.png
Matrice sauvegardee: C:\Users\Nouhaila\Projet_Yoga_AI\results\plots\confusion_avec_fond_original___knn.png
Matrice sauvegardee: C:\Users\Nouhaila\Projet_Yoga_AI\results\plots\confusion_sans_fond_traitee___svm.png
Matrice sauvegardee: C:\Users\Nouhaila\Projet_Yoga_AI\results\plots\confusion_sans_fond_traitee___knn.png
Graphique comparatif sauvegarde: C:\Users\Nouhaila\Projet_Yoga_AI\results\plots\comparaison_fond.png
Tableau comparatif sauvegarde: C:\Users\Nouhaila\Projet_Yoga_AI\results\plots\comparaison_modeles_table.png
```

Figure 11: Résultat du terminal

2. Page Web (en exécutant streamlit run src/web_app.py) :

La page web est un tableau de bord interactif et visuel qui offre une exploration plus riche et int uitive des résultats du projet. Elle est divisée en trois sections principales :

- Dashboard (Tableau de Bord):
  - Indicateurs Clés : Des cartes affichent les métriques importantes comme la meilleure précision obtenue et le gain moyen de performance grâce à la suppression du fond.
  - Graphique Comparatif : Un graphique à barres interactif permet de visualiser et de comparer facilement la précision des modèles (SVM vs KNN) pour les deux expériences (avec et sans fond).
  - Tableau Détaillé : Un tableau complet présente les données de performance pour chaque configuration entraînée.
- Pose Detection (Détection de Pose) :
  - Prédiction en Direct : Cette section vous permet de téléverser votre propre image d'une posture de yoga.
  - Sélection du Modèle : Vous pouvez choisir quel modèle entraîné utiliser pour la prédiction.
  - Affichage des Résultats : Après avoir cliqué sur "Prédire la pose", l'application affiche l'image, la posture prédite, et le niveau de confiance en pourcentage. Un graphique montre également les scores de confiance pour toutes les autres postures.
- Model Insights (Analyse des Modèles) :
  - Analyses Approfondies : Vous pouvez sélectionner une expérience spécifique pour l'analyser en détail.
  - Courbes d'Apprentissage : Des graphiques montrent l'évolution de la précision (Accuracy) et de l'erreur (Loss) pendant l'entraînement, ce qui aide à diagnostiquer le sur-apprentissage ou le sous-apprentissage.
  - Matrice de Confusion : Une visualisation interactive de la matrice de confusion montre quelles postures sont souvent confondues entre elles.

- Métriques par Classe : Des graphiques et un tableau détaillent la précision, le rappel et le score F1 pour chaque posture de yoga, permettant d'identifier les forces et faiblesses du modèle.

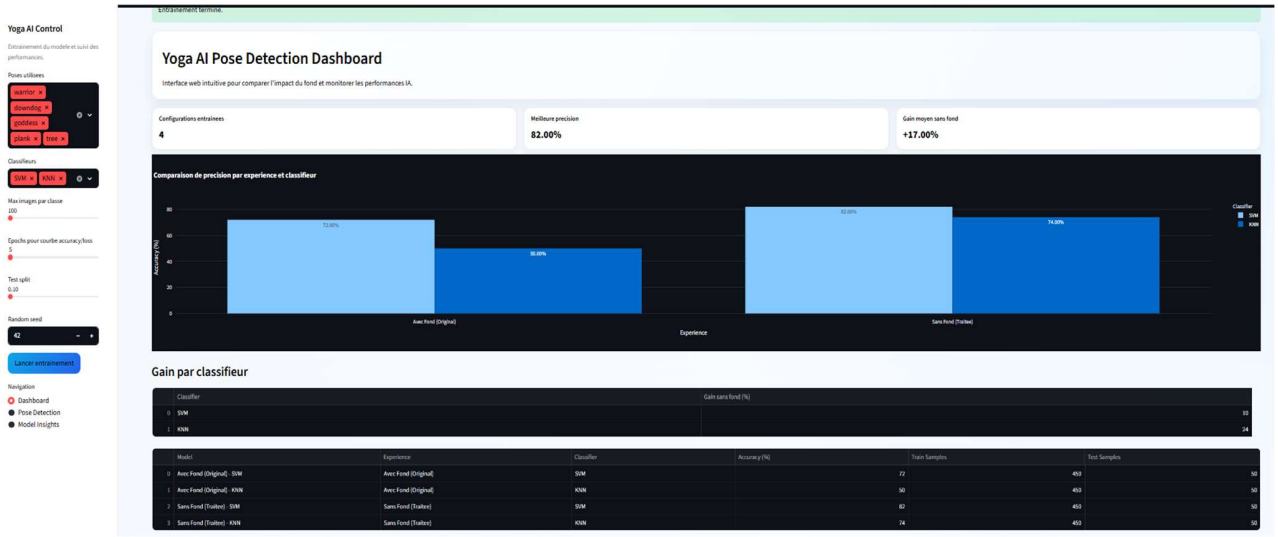

Figure 12: Résultat de l'entrainement sur la page web

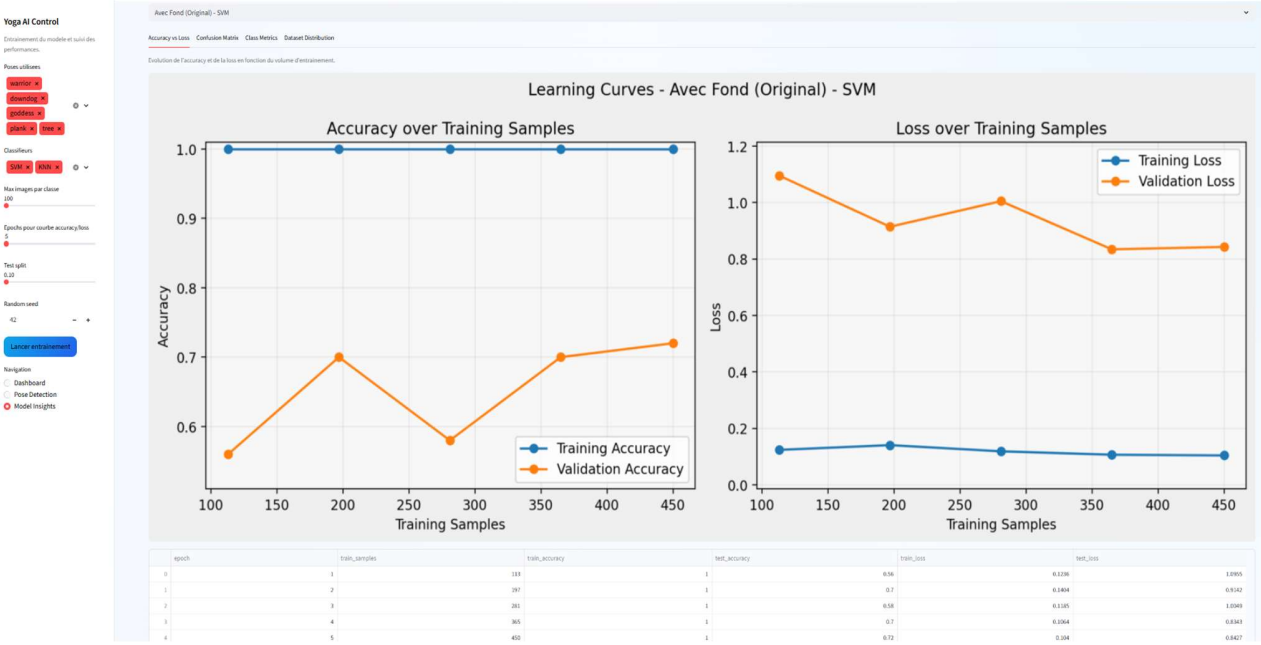

Figure 13: Les diagrammes

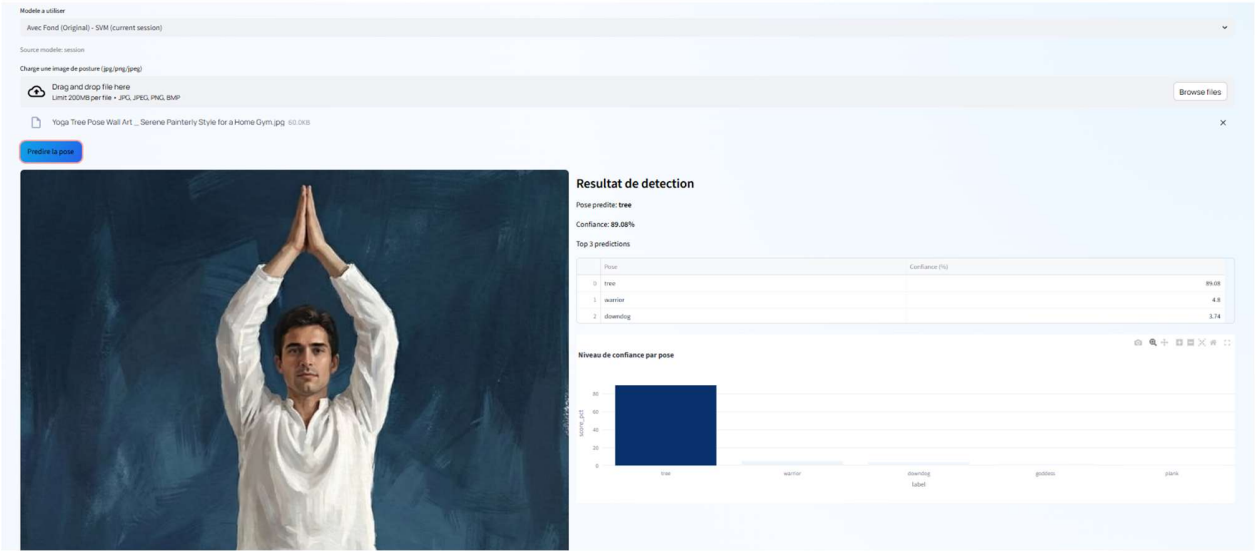

Figure 14: Prédiction des poses

3. Conclusion :

Ce projet met en œuvre avec succès un système de classification de postures de yoga en utilisant des techniques de vision par ordinateur (HOG) et d'apprentissage automatique (SVM, KNN). La conclusion la plus significative est l'impact positif et quantifiable de la suppression de l'arrière-plan, qui améliore nettement la précision des modèles. Cela démontre l'importance cruciale du prétraitement des données dans les tâches de vision par ordinateur. Le terminal sert d'outil rapide pour valider les performances et générer les résultats finaux, tandis que l'application web transforme le projet en un outil d'expérimentation dynamique et accessible. Elle permet non seulement de tester interactivement les modèles, mais aussi de comprendre en profondeur leur comportement, rendant l'analyse accessible même à un public moins technique.